Step 1. Analyze the Data

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

ds = pd.read_csv("wan_dataset.csv")

print(ds.head())
print(ds.info())
print(ds.describe())
```

```
2          0  link_3        host_3           switch_1      interface_3
3          0  link_4        host_4           switch_3      interface_4
4          0  link_5        host_5           switch_1      interface_5

  destination_interface  link_utilization  latency_ms  packet_loss  \
0           interface_1                50          35          0.0
1           interface_4                83          28          1.2
2           interface_7                86          13          0.4
3          interface_10                55           5          1.6
4           interface_1                19          10          0.8

   throughput_mbps  qos_priority  reachability   traffic_type  anomaly_flag
0              200             1             0           http             1
1              452             2             1          video             0
2              472             3             1           voip             0
3              242             4             1  file_transfer             0
4              -27             5             1           http             0
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 14 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   timestamp             2000 non-null   int64
 1   link_id               2000 non-null   object
 2   source_device         2000 non-null   object
 3   destination_device    2000 non-null   object
 4   source_interface      2000 non-null   object
 5   destination_interface  2000 non-null   object
 6   link_utilization      2000 non-null   int64
 7   latency_ms            2000 non-null   int64
 8   packet_loss           2000 non-null   float64
 9   throughput_mbps       2000 non-null   int64
 10  qos_priority          2000 non-null   int64
 11  reachability          2000 non-null   int64
 12  traffic_type          2000 non-null   object
 13  anomaly_flag          2000 non-null   int64
dtypes: float64(1), int64(7), object(6)
memory usage: 218.9+ KB
None
         timestamp  link_utilization   latency_ms  packet_loss  \
count  2000.000000       2000.000000  2000.000000  2000.000000
mean     99.500000         49.905000    18.941500     0.981000
std      57.748744         28.220067    10.552334     0.577413
min       0.000000         10.000000     5.000000     0.000000
25%      49.750000         22.000000     8.000000     0.480000
50%      99.500000         50.000000    18.000000     1.000000
75%     149.250000         78.000000    29.250000     1.480000
max     199.000000         89.000000    35.000000     2.000000

       throughput_mbps  qos_priority  reachability  anomaly_flag
count      2000.000000   2000.000000   2000.000000   2000.000000
mean        204.797000      3.000000      0.960000      0.020000
std         211.489704      1.414567      0.196008      0.140035
min         -99.000000      1.000000      0.000000      0.000000
25%          -7.000000      2.000000      1.000000      0.000000
50%         210.000000      3.000000      1.000000      0.000000
75%         417.000000      4.000000      1.000000      0.000000
max         499.000000      5.000000      1.000000      1.000000
```

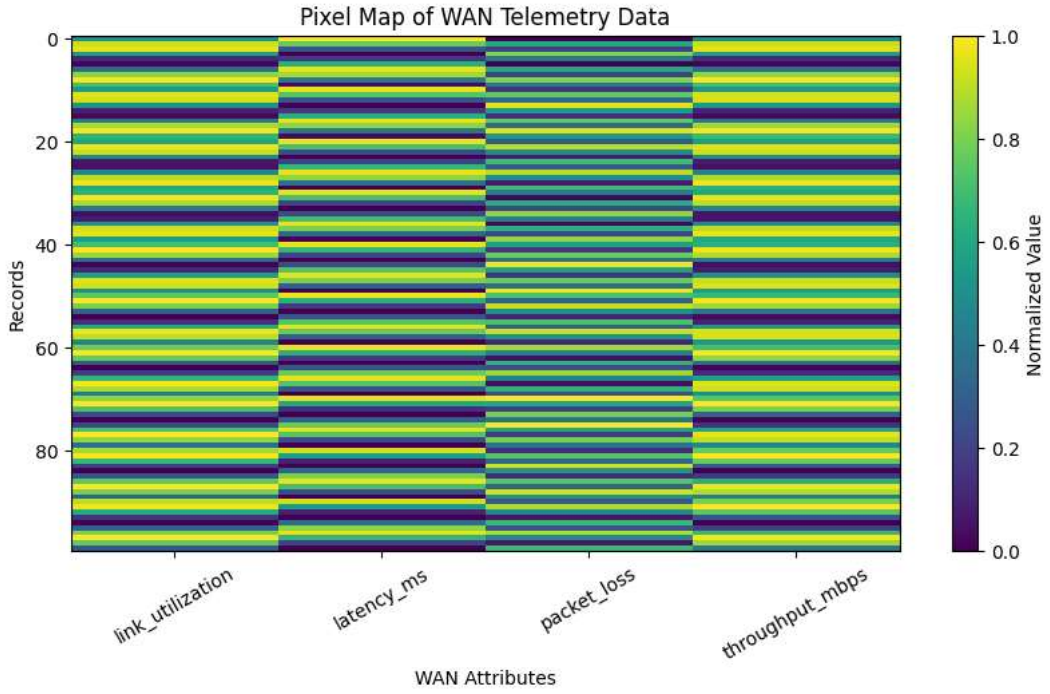
Step 2. Pixel-Oriented Visualization

A pixel map represents individual records using colours. The heatmap shows average WAN metrics for each link.

```
pixel_data = ds[['link_utilization', 'latency_ms', 'packet_loss', 'throughput_mbps']].head(100)

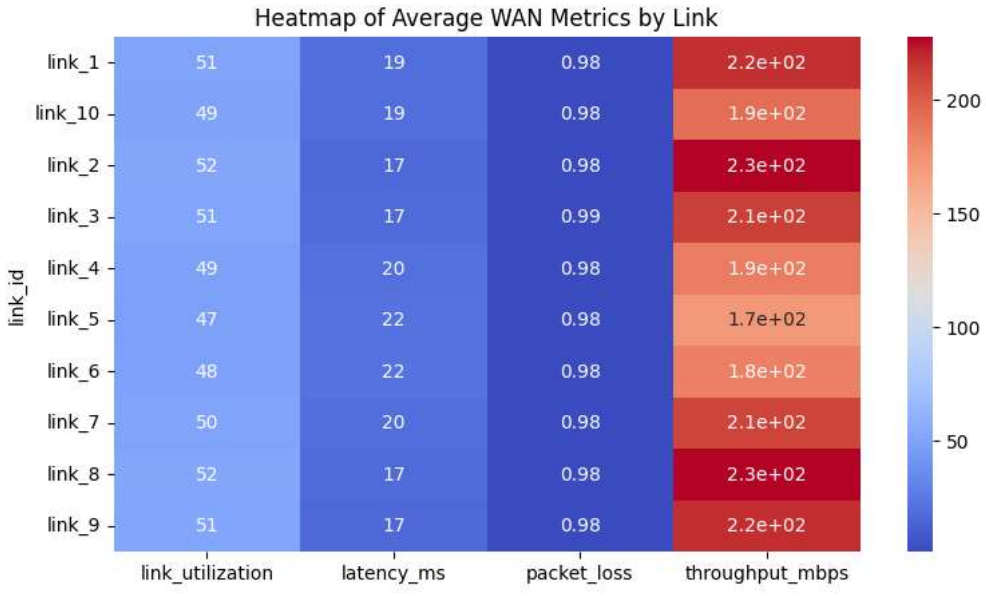
pixel_data = (pixel_data - pixel_data.min()) / (pixel_data.max() - pixel_data.min())

plt.figure(figsize=(10, 5))
plt.imshow(pixel_data, aspect='auto', cmap='viridis')
plt.colorbar(label="Normalized Value")
plt.title("Pixel Map of WAN Telemetry Data")
plt.xlabel("WAN Attributes")
plt.ylabel("Records")
plt.xticks(range(4), pixel_data.columns, rotation=30)
plt.show()
```




```
heat_data = ds.groupby('link_id')[['link_utilization', 'latency_ms', 'packet_loss', 'throughput_mbps']].mean()

plt.figure(figsize=(9, 5))
sns.heatmap(heat_data, annot=True, cmap='coolwarm')
plt.title("Heatmap of Average WAN Metrics by Link")
plt.show()
```



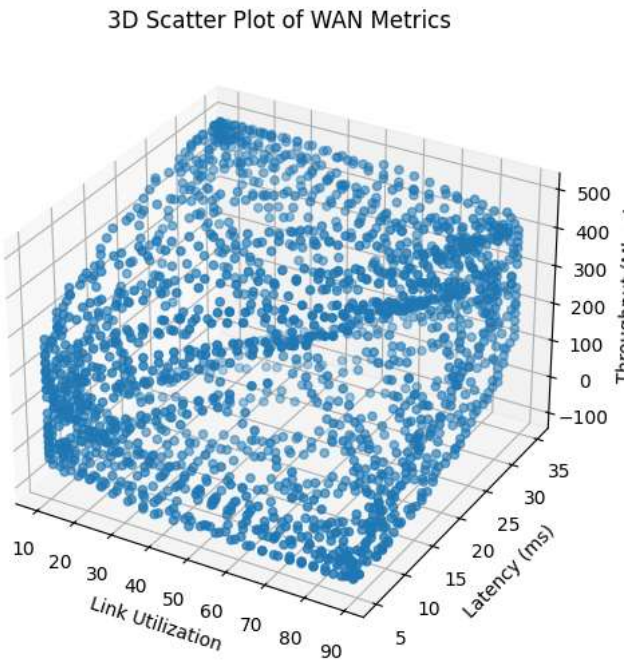
Step 3. Geometric Projection Visualization

The first plot displays three WAN attributes in 3D. The second uses three 2D orthographic views: XY, XZ and YZ.

```
fig = plt.figure(figsize=(9, 6))
ax = fig.add_subplot(111, projection='3d')

ax.scatter(ds['link_utilization'], ds['latency_ms'], ds['throughput_mbps'])

ax.set_title("3D Scatter Plot of WAN Metrics")
ax.set_xlabel("Link Utilization")
ax.set_ylabel("Latency (ms)")
ax.set_zlabel("Throughput (Mbps)")
plt.show()
```




```
fig, ax = plt.subplots(1, 3, figsize=(15, 4))

ax[0].scatter(ds['link_utilization'], ds['latency_ms'])
ax[0].set_title("XY Projection")
ax[0].set_xlabel("Link Utilization")
ax[0].set_ylabel("Latency")

ax[1].scatter(ds['link_utilization'], ds['throughput_mbps'])
ax[1].set_title("XZ Projection")
ax[1].set_xlabel("Link Utilization")
ax[1].set_ylabel("Throughput")

ax[2].scatter(ds['latency_ms'], ds['throughput_mbps'])
ax[2].set_title("YZ Projection")
ax[2].set_xlabel("Latency")
ax[2].set_ylabel("Throughput")

plt.tight_layout()
plt.show()
```

Step 4. Icon-Based Visualization

Sparklines show small trends, glyphs use marker size and colour to represent values, and an icon array displays normal and anomalous records.

```
plt.figure(figsize=(10, 4))
```

```
for link in ds['link_id'].unique()[:5]:
    link_data = ds[ds['link_id'] == link].head(30)
    plt.plot(range(len(link_data)), link_data['latency_ms'], label=str(link))

plt.title("Sparklines: Latency Trend for Links")
plt.xlabel("Record Number")
plt.ylabel("Latency (ms)")
plt.legend(title="Link ID")
plt.show()
```

```
plt.figure(figsize=(8, 5))
plt.scatter(ds['link_utilization'], ds['latency_ms'],
            s=ds['throughput_mbps'], c=ds['packet_loss'], alpha=0.5)
plt.title("Glyph Visualization of WAN Metrics")
plt.xlabel("Link Utilization")
plt.ylabel("Latency (ms)")
plt.colorbar(label="Packet Loss")
plt.show()
```

```
anomaly_count = ds['anomaly_flag'].value_counts().sort_index()

plt.figure(figsize=(7, 3))
plt.bar(anomaly_count.index.astype(str), anomaly_count.values)
plt.title("Icon Array: Normal and Anomalous Records")
plt.xlabel("Anomaly Flag (0 = Normal, 1 = Anomaly)")
plt.ylabel("Number of Records")
plt.show()
```

Step 5. Hierarchical Visualization

Treemap and sunburst charts show records grouped by traffic type and anomaly flag. The dendrogram groups links based on their average WAN metrics.

```
import plotly.express as px

tree_data = ds.groupby(['traffic_type', 'anomaly_flag']).size().reset_index(name='records')
tree_data['anomaly_flag'] = tree_data['anomaly_flag'].astype(str)

fig = px.treemap(tree_data, path=['traffic_type', 'anomaly_flag'],
                 values='records', title="Treemap of Traffic Type and Anomaly")
fig.show()
```

```
fig = px.sunburst(tree_data, path=['traffic_type', 'anomaly_flag'],
                  values='records', title="Sunburst of Traffic Type and Anomaly")
fig.show()
```

```
from scipy.cluster.hierarchy import dendrogram, linkage

link_data = ds.groupby('link_id')[['link_utilization', 'latency_ms',
                                   'packet_loss', 'throughput_mbps']].mean()

cluster_data = linkage(link_data, method='ward')

plt.figure(figsize=(10, 5))
dendrogram(cluster_data, labels=link_data.index.astype(str))
plt.title("Dendrogram of WAN Links")
plt.xlabel("Link ID")
plt.ylabel("Distance")
plt.show()
```

Step 6. Explore a Data Visualization Tool

Plotly is used here to create an interactive scatter plot. You can zoom, hover over points and explore the values.

```
fig = px.scatter(ds, x='link_utilization', y='latency_ms',
                 color='traffic_type', hover_data=['link_id', 'packet_loss'],
                 title="Interactive WAN Telemetry Visualization")
fig.show()
```